

	SR UNIVERSITY	
	Campus Warangal	
	Program: II - B. Tech (CS& AI) Batch:	
	Name: Alok Ranjan	Enroll.No:2503A54050
	Department: Computer Science and AI	Semester: I
	AI Assisted Coding-Assignment	Date: 18-09-26

Q1. Class with Conditional Logic – Employee Salary

Scenario:

An AI coding assistant is used to develop a simple employee salary management program.

Task:

Using AI-assisted code completion, write a Python class Employee that:

- Stores employee name and basic salary.
- Has a method calculate_bonus() that:
 - Returns 10% of salary if salary is ₹50,000 or above.
 - Returns 5% of salary otherwise.
- Has a method display_details() to display the employee name, salary, and bonus.

Create at least two employee objects and test the class.

1. Prompt

Generate a complete, executable Python program for an Employee

Salary Calculator using Object-Oriented Programming.

Requirements:

1. Create a class named Employee.
2. The class must have the attributes:
 - name
 - basic_salary

3. Create a method named `calculate_bonus()`.
4. If `basic_salary` is greater than or equal to 50000, calculate a bonus of 10% of the basic salary.
5. If `basic_salary` is less than 50000, calculate a bonus of 5% of the basic salary.
6. Create a method named `calculate_total_salary()` that returns `basic_salary + bonus`.
7. Create at least two `Employee` objects with different salary values.
8. Display the following for each employee:
 - Employee Name
 - Basic Salary
 - Bonus
 - Total Salary
9. Use appropriate conditional statements.
10. Use meaningful variable and method names.
11. Add brief comments explaining the important parts of the code.
12. Do not use external libraries.
13. The program must run without errors.

After generating the code, provide three sample test cases, including:

- salary below 50000
- salary exactly 50000
- salary above 50000

Also show the expected output for each test case.

CODE:-

```
ai_asist_7.py X jbjdv.py X
jbjdv.py > ...
1 # Employee Salary Calculator using Object-Oriented Programming
2
3 class Employee:
4     # Constructor to initialize employee details
5     def __init__(self, name, basic_salary):
6         self.name = name
7         self.basic_salary = basic_salary
8
9     # Method to calculate bonus based on basic salary
10    def calculate_bonus(self):
11        if self.basic_salary >= 50000:
12            return self.basic_salary * 0.10
13        else:
14            return self.basic_salary * 0.05
15
16    # Method to calculate total salary
17    def calculate_total_salary(self):
18        bonus = self.calculate_bonus()
19        return self.basic_salary + bonus
20
21
22 # Create Employee objects with different salary values
23 employee1 = Employee("Rahul", 45000)
24 employee2 = Employee("Priya", 60000)
25
26 # Display details of each employee
27 for employee in [employee1, employee2]:
28     bonus = employee.calculate_bonus()
29     total_salary = employee.calculate_total_salary()
30
31     print("Employee Name :", employee.name)
32     print("Basic Salary : ₹", employee.basic_salary)
33     print("Bonus : ₹", bonus)
34     print("Total Salary : ₹", total_salary)
35     print("-" * 35)
```

OUTPUT:-

```
04-python.exe C:/Users/alokr/Desktop/AI-assisted/jogo
Employee Name : Rahul
Basic Salary  : ₹ 45000
Bonus        : ₹ 2250.0
Total Salary  : ₹ 47250.0
-----
Employee Name : Priya
Basic Salary  : ₹ 60000
Bonus        : ₹ 6000.0
Total Salary  : ₹ 66000.0
-----
PS C:\Users\alokr\Desktop\AI-assisted> █
```

Observation:-

The program successfully implements the Employee class using conditional logic. Employees earning **₹50,000 or more receive a 10% bonus**, while employees earning below ₹50,000 receive a **5% bonus**. The total salary is calculated by adding the bonus to the basic salary.

Q2. Loop and Conditional Logic – Student Marks

Scenario:

A teacher wants to analyze the marks of students using an AI coding assistant.

Task:

Using AI-assisted code completion, write a Python program that:

- Accepts a list of student marks.
- Uses a loop to process each mark.
- Displays:
 - "Distinction" for marks ≥ 75
 - "Pass" for marks ≥ 40 and < 75
 - "Fail" for marks < 40
- Ignores invalid marks outside 0–100.
- Calculates and displays the number of students in each category.

Test the program with a suitable list of marks.

PROMPT:-

Generate a complete Python program that accepts a list of student marks and uses a loop and conditional statements to classify each student.

Requirements:

1. Marks of 75 or above should be displayed as "Distinction".
2. Marks from 40 to below 75 should be displayed as "Pass".
3. Marks below 40 should be displayed as "Fail".
4. Invalid marks outside the range 0–100 should be ignored.
5. Count and display the number of students in each category.
6. Test the program using a suitable list of marks.

CODE:-

```
38
39 marks = [85, 72, 35, 90, 48, 105, -5, 65, 25]
40
41 distinction = 0
42 passed = 0
43 failed = 0
44
45 for mark in marks:
46     if mark < 0 or mark > 100:
47         continue
48
49     if mark >= 75:
50         print(mark, "-> Distinction")
51         distinction += 1
52     elif mark >= 40:
53         print(mark, "-> Pass")
54         passed += 1
55     else:
56         print(mark, "-> Fail")
57         failed += 1
58
59 print("\nNumber of Distinction:", distinction)
60 print("Number of Pass:", passed)
61 print("Number of Fail:", failed)
```

Output:-

```
85 -> Distinction
72 -> Pass
35 -> Fail
90 -> Distinction
48 -> Pass
65 -> Pass
25 -> Fail

Number of Distinction: 2
Number of Pass: 3
Number of Fail: 2
PS C:\Users\alokr\Desktop\AI assisted> █
```

Observation:-

The program uses a **loop** to process every mark and conditional statements to classify students. Marks of 75 or above are classified as Distinction, marks from 40 to below 75 as Pass, and marks below 40 as Fail. Invalid marks outside 0–100 are ignored.

Q3. Class + Loop – Product Inventory

Scenario:

A small store wants to maintain product inventory using a Python program generated with AI assistance.

Task:

Using AI-assisted code completion, create a class Product that:

- Stores product name, price, and quantity.
- Has a method `total_value()` that calculates $\text{price} \times \text{quantity}$.
- Create a list of multiple Product objects.
- Use a loop to calculate the total inventory value.
- Display the products whose quantity is less than 5 as "Low Stock".

Test the program with at least three products.

Prompt:-

Generate a complete Python program using a Product class for inventory management.

Requirements:

1. Create a class named Product.
2. Store product name, price, and quantity.
3. Create a method `total_value()` that calculates price × quantity.
4. Create a list containing multiple Product objects.
5. Use a loop to calculate the total inventory value.
6. Display products whose quantity is less than 5 as "Low Stock".
7. Test the program with at least three products.

Code:-

```
64
65 class Product:
66     def __init__(self, name, price, quantity):
67         self.name = name
68         self.price = price
69         self.quantity = quantity
70
71     def total_value(self):
72         return self.price * self.quantity
73
74
75 products = [
76     Product("Laptop", 50000, 3),
77     Product("Mouse", 800, 10),
78     Product("Keyboard", 1500, 4)
79 ]
80
81 total_inventory = 0
82
83 for product in products:
84     value = product.total_value()
85     total_inventory += value
86
87     print("Product :", product.name)
88     print("Price   :", product.price)
89     print("Quantity:", product.quantity)
90     print("Value   :", value)
91
92     if product.quantity < 5:
93         print("Status : Low Stock")
94
95     print()
96
97 print("Total Inventory Value:", total_inventory)
```

Output:-

```
64\python.exe "c:/Users/alokr/Desktop/AI assisted/AI_Assist_7.py"
Product : Laptop
Price   : 50000
Quantity: 3
Value   : 150000
Status  : Low Stock

Product : Mouse
Price   : 800
Quantity: 10
Value   : 8000

Product : Keyboard
Price   : 1500
Quantity: 4
Value   : 6000
Status  : Low Stock

Total Inventory Value: 164000
PS C:\Users\alokr\Desktop\AI assisted>
```

Observation:-

The program successfully uses a Product class to store product information. The `total_value()` method calculates price × quantity, while a loop calculates the total inventory value. Products having a quantity below 5 are correctly identified as Low Stock.

Q4. Class + Loop + Nested Conditions – Electricity Bill

Scenario:

An electricity board wants to calculate bills based on units consumed.

Task:

Using AI-assisted code completion, create a Python class `ElectricityBill` that:

- Stores the consumer name and units consumed.
- Calculates the bill using:
 - First 100 units → ₹2/unit

- 101–200 units → ₹3/unit
 - Above 200 units → ₹5/unit
- Uses appropriate **loops and conditional statements** where required.
- Displays the consumer name, units, and total bill.
- Create at least three objects and display their bills.
- Test the program with different unit values, including an edge value such as 100 or 200.

Prompt:-

Generate a complete Python program using an Electricity Bill class to calculate electricity bills.

Requirements:

1. Create a class named Electricity Bill.
2. Store consumer name and units consumed.
3. Calculate the bill using:
 - First 100 units at ₹2 per unit.
 - Units 101–200 at ₹3 per unit.
 - Units above 200 at ₹5 per unit.
4. Use appropriate conditional statements.
5. Display consumer name, units consumed, and total bill.
6. Create at least three objects.
7. Test different unit values, including edge values such as 100 and 200.

Code:-

```

class ElectricityBill:
    def __init__(self, name, units):
        self.name = name
        self.units = units

    def calculate_bill(self):
        if self.units <= 100:
            return self.units * 2

        elif self.units <= 200:
            return (100 * 2) + ((self.units - 100) * 3)

        else:
            return (100 * 2) + (100 * 3) + ((self.units - 200) * 5)

    def display_bill(self):
        bill = self.calculate_bill()

        print("Consumer Name :", self.name)
        print("Units Consumed:", self.units)
        print("Total Bill      : ₹", bill)
        print()

consumer1 = ElectricityBill("Rahul", 100)
consumer2 = ElectricityBill("Priya", 200)
consumer3 = ElectricityBill("Amit", 250)

consumer1.display_bill()
consumer2.display_bill()
consumer3.display_bill()

```

Output:-

```

Consumer Name : Rahul
Units Consumed: 100
Total Bill      : ₹ 200

Consumer Name : Priya
Units Consumed: 200
Total Bill      : ₹ 500

Consumer Name : Amit
Units Consumed: 250
Total Bill      : ₹ 750

```

PS C:\Users\alokr\Desktop\AI assisted> █

Observation:-

The program successfully calculates electricity bills according to the given unit ranges. Conditional statements are used to apply the appropriate rate for each range. The edge values 100 and 200 units are also tested successfully.